

Envertech EVT2000SE – Protokollspezifikation

Technische Beschreibung des lokalen TCP-Protokolls, basierend auf:

- Decompiled APK der EnverView-App (v4.1.20, `com.jialeinfo.enver`)
- Live-Tests gegen einen EVT2000SE (Firmware 164.125)
- PCAP-Mitschnitte

Erstellt von SyntaxSorcerer7

Übersicht

Eigenschaft	Wert
Transport	TCP
TCP-Port	14889
TX-Encoding (Client → WR)	ASCII-HEX
RX-Encoding (WR → Client)	Binär
Startbyte	0x68
Endbyte	0x16
Prüfsumme	(Bytesumme + 0x55) & 0xFF
Sicherheit	Keine (kein TLS, kein Auth)

Gerätearchitektur

Der EVT2000SE enthält **vier logische Mikroinverter (MI)** mit fortlaufenden UUIDs:

UUID	MI
0xA1B2C3D4	MI 0 (= Geräte-Serial)
0xA1B2C3D5	MI 1
0xA1B2C3D6	MI 2
0xA1B2C3D7	MI 3

UUIDs sind Hex-Werte. Implementierung: `struct.pack(">I", 0xA1B2C3D4)`

Asymmetrisches Encoding

TX (Client → WR): Frame als ASCII-Hex-String senden. Jedes Byte → 2 ASCII-Zeichen.

Byte 0x68 → ASCII "68" → TCP-Bytes 0x36 0x38

RX (WR → Client): Rohe Binärbytes.

Prüfsumme wird immer über die Binärbytes berechnet.

Frameformat

[68] [LEN_H] [LEN_L] [68] [CMD_H] [CMD_L] [SERIAL 4B] [PAYLOAD ...] [CS]
[16]

Feld	Bytes	Beschreibung
0x68	1	Startbyte
LEN	2	Gesamtlänge des Frames (Big-Endian)
0x68	1	Zweites Startbyte
CMD	2	Command-ID (Big-Endian)
SERIAL	4	WR-Seriennummer (Big-Endian)
PAYLOAD	variabel	Nutzdaten
CS	1	Prüfsumme
0x16	1	Endbyte

Prüfsumme

```
def checksum(data: bytes) -> int:  
    return (sum(data) + 0x55) & 0xFF
```

Eingabe: Alle Bytes von erstem 0x68 bis einschließlich letztem Payload-Byte (= alles außer CS und Endbyte).

Verbindungsstrategie (verifiziert)

Empfehlung: Eine Verbindung offen halten

Strategie	Erfolgsrate	Response-Zeit
Gleiche Verbindung wiederverwenden	100%	~670ms
Neue Verbindung pro Anfrage (3s Pause)	~40%	~700ms
Neue Verbindung (0s Pause)	unzuverlässig	–

Der WR wird bei häufigem Neuverbinden instabil. Auf einer bestehenden Verbindung antwortet er dagegen zuverlässig – auch nach 15s Idle.

Timing-Parameter

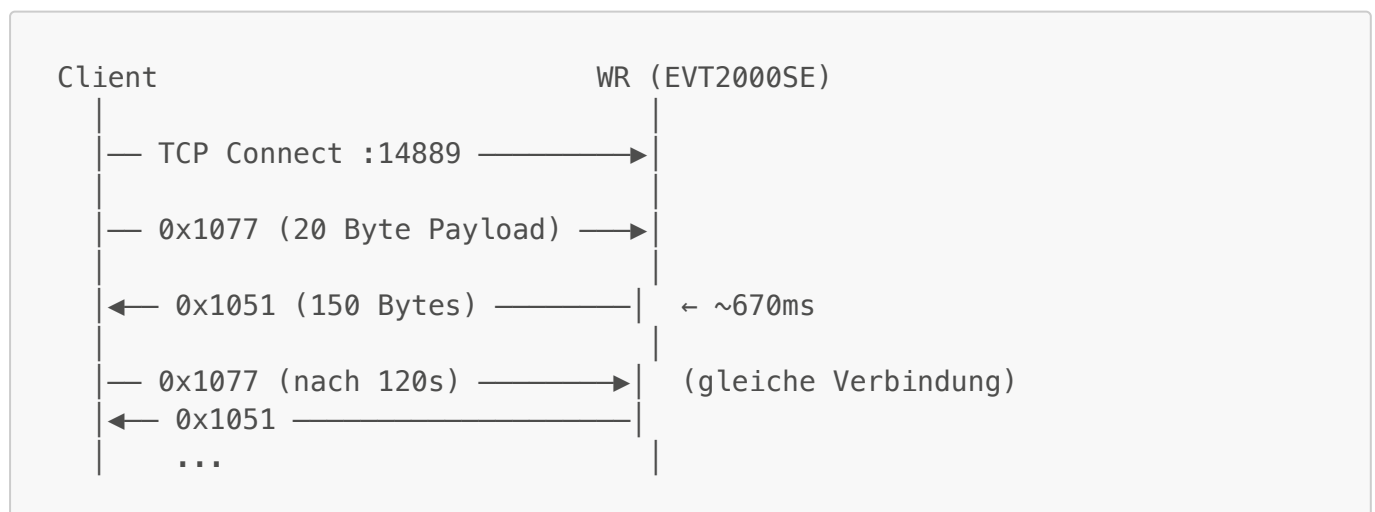
Parameter	Wert
TCP-Connect-Timeout	5s
Response-Timeout (0x1077)	3s
Response-Timeout (0x1041)	10s
Retry-Delay bei Fehlschlag	3s
Max Retries 0x1077	3
Max Retries 0x1041	5 (mit 5s Delay)
Poll-Intervall (wie App)	120s

Parallele Verbindungen

Funktionieren – der WR beantwortet mehrere gleichzeitige TCP-Verbindungen.

Kommunikationsablauf (Gerätetyp INVERTER)

Live-Daten lesen (0x1077 → 0x1051)

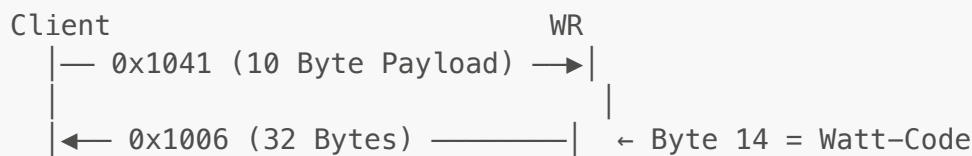


Für den INVERTER-Typ (EVT2000SE direkt) kommt **nur 0x1051** – keine 0x1004-Frames davor.

Empfangs-Logik

```
sock.sendall(encode_tx(build_frame(0x1077, serial, bytes(20))))
while True:
    frame = receive_frame(sock, timeout=3)
    cmd = get_cmd(frame)
    if cmd == 0x1051:
        return parse_live_data(frame)
    elif cmd in (0x1004, 0x1006):
        continue # Überspringen falls sie kommen
    elif frame is None:
        raise TimeoutError("Retry nötig")
```

Aktuelle Leistungsgrenze lesen (0x1041 → 0x1006)



Unzuverlässig (~20% Erfolgsrate bei neuer Verbindung). Retry-Strategie: bis zu 5 Versuche mit 5s Delay.

Leistungsgrenze setzen (0x1137)



Wird direkt ohne Session-Init gesendet.

0x1051-Payload – Live Data Response (150 Bytes)

Header

Offset	Feld	Typ
10	Firmware-Version (Haupt)	uint8
12	Firmware-Version (Neben)	uint8
14	E-Meter Typ	0=keins, 1=einphasig, 3=dreiphasig

Offset	Feld	Typ
15	MI-Phase	0=nicht gesetzt, 1=A, 2=B, 3=C
16	Anti-Rückspeisung	0=aus, 1=ein
17	Anti-Rückspeisung Anzeige	Enum

Kanal-Blöcke (4x, je 32 Bytes, ab Offset 20)

Offset (relativ)	Feld	Formel
0–3	UID	uint32 BE
6–7	DC-Spannung	$\text{uint16} \times 64 / 32768 \text{ V}$
8–9	AC-Leistung	$\text{uint16} \times 512 / 32768 \text{ W}$
10–13	Gesamtenergie	$\text{uint32} \times 4 / 32768 \text{ kWh}$
14–15	Temperatur	$\text{uint16} \times 256 / 32768 - 40 \text{ °C}$
16–17	AC-Spannung	$\text{uint16} \times 512 / 32768 \text{ V}$
18–19	Frequenz	$\text{uint16} \times 128 / 32768 \text{ Hz}$

Verifizierte Beispielwerte (Sonnentag, Mittag)

MI	DC-V	AC-W	kWh	Temp	AC-V	Freq
0	32.6	192.1	22.42	48.7°C	235.6	50.00
1	31.7	198.0	21.24	50.5°C	235.6	50.00
2	34.0	179.2	18.04	51.4°C	235.6	50.00
3	35.0	133.3	13.95	47.7°C	235.6	50.00

Python

```
def parse_channel(block): # 32 Bytes
    uid = struct.unpack(">I", block[0:4])[0]
    dc_v = struct.unpack(">H", block[6:8])[0] * 64 / 32768.0
    ac_w = struct.unpack(">H", block[8:10])[0] * 512 / 32768.0
    kwh = struct.unpack(">I", block[10:14])[0] * 4 / 32768.0
    temp = struct.unpack(">H", block[14:16])[0] * 256 / 32768.0 - 40.0
    ac_v = struct.unpack(">H", block[16:18])[0] * 512 / 32768.0
    freq = struct.unpack(">H", block[18:20])[0] * 128 / 32768.0
    return uid, dc_v, ac_w, kwh, temp, ac_v, freq
```

0x1006-Payload – Status/Heartbeat (32 Bytes)

Kommt als Antwort auf **0x1041**, auch nach **0x1137**-SET-Commands.

Offset	Feld	Beschreibung
10	Unbekannt	Immer 0x00
14	Watt-Code	Aktuell eingestellte Leistungsgrenze

Verifiziertes Beispiel (1200W eingestellt):

```
68 00 20 68 10 06 A1 B2 C3 D4 00 00 00 00 01 00 4b 00 00 c6 03 00 00 01 05
00 01 00 00 00 ed 16
```

```
          ^^          ^^
          Byte10=0x00  Byte14=0x01 → 1200W
```

Leistungsbegrenzung

0x1137 – Feste Watt-Stufen

Setzt eine diskrete Watt-Begrenzung. **0x00** = volle Nennleistung.

EVT2000SE Watt-Codes

Watt	Code
600	0x09
800	0x0A
1200	0x01
1400	0x0B
1440	0x02
1600	0x06
1640	0x0C
1800	0x07
2000	0x00

Andere Modellvarianten

Variante	Werte	Codes
"4" (400W)	150, 200, 300, 350, 360, 400 W	0x09, 0x0A, 0x01, 0x0B, 0x02, 0x00
"5" (800W)	300, 400, 600, 700, 720, 800 W	0x09, 0x0A, 0x01, 0x0B, 0x02, 0x00

Variante	Werte	Codes
"7" (1000W)	300, 400, 600, 700, 720, 800, 820, 900, 1000 W	0x09, 0x0A, 0x01, 0x0B, 0x02, 0x03, 0x0C, 0x0D, 0x00

Aktuellen Watt-Code lesen

Es gibt **keinen** dedizierten Read-Command. Der aktuelle Watt-Code steht in **0x1006** Byte 14, ausgelöst durch:

- **0x1041** senden → **0x1006** empfangen (unzuverlässig, Retries nötig)
- Nach **0x1137**-SET folgt ebenfalls **0x1006**

0x1174 / 0x1175 – Prozentuale Begrenzung

Command	Richtung	Beschreibung
0x1174	TX (Set)	Active Power Percentage setzen
0x1175	TX (Get)	Abfrage – antwortet nie (verifiziert)

Gesendet wird ein Index (0–19):

Index	Prozent
0	5%
1	10%
...	+5% pro Schritt
19	100%

UDP Discovery (optional)

Dient nur zum Finden der IP-Adresse. Bei bekannter IP überspringen.

Ethernet: UDP-Broadcast an Port **48889**, Payload "**LOCALCON-1508-READ**". Antwort: Bytes 0–3 = IP, Bytes 6–9 = Serial.

WiFi: UDP an **10.10.100.254:48899**, Payload "**www.usr.cn**" (USR-IOT Modul).

Vollständige Command-Liste

Kernfunktionen

TX-Cmd	RX-Cmd	Beschreibung
0x1077	0x1051	Live Data – Payload: 20 Nullbytes. Antwort: 150 Bytes mit 4 MI-Kanalblöcken.
0x1041	0x1006	Disconnect / Heartbeat-Trigger – <code>breakSendByte()</code> . Payload: 10 Nullbytes. Antwort: 32B, Byte 14=Watt-Code. Unzuverlässig.
0x1137	0x1137	Power Limit Set – Payload: 1 Byte Watt-Code. Antwort: 32B, Byte 10=bestätigter Code.
0x1039	0x1040	Monitor Init – Nur für MONITOR-Typ (Gateway). Nicht nötig bei EVT2000SE direkt.
0x1078	0x1079	UID-Liste – Nur MONITOR-Typ. MI-Zuordnung zu Phasen A/B/C.

Historie

TX-Cmd	RX-Cmd	Beschreibung
0x1042	0x1043	Zeitsync – Jahr-1900, Monat, Tag, H, M, S.
0x1044	0x1045	Historie abrufen – Start-/Enddatum. 982B-Blöcke, 30 Datensätze je 32B.
0x1046	0x1047	Frühestes Datum – Bytes 14–16: Jahr+1900, Monat, Tag.
0x1111	–	Transfer-Ende – Alle Historien-Blöcke übertragen.

E-METER und Nulleinspeisung

Hardware

Proprietäres WiFi-Smart-Meter (Modell MW41B), basiert auf USR-WiFi-Modul. Kommuniziert per WiFi – kein RS485/Modbus/CT-Kabel zum WR nötig.

E-Meter Typen (Byte 14 in 0x1051)

Wert	Bedeutung
0	Kein Meter
1	Einphasig
3	Dreiphasig

Pairing

Meter erzeugt Hotspot (`SmartMeter*/MW41B*`). App sendet WLAN-Credentials per UDP an `10.10.100.254:48899`:


```
AT+WSSSID=<SSID>
AT+WSKEY=WPA2PSK,AES,<Passwort>
AT+WANN    (DHCP)
AT+Z      (Neustart)
```

Datenweg

Wahrscheinlich Cloud-vermittelt: Meter sendet an Envertech-Cloud, WR ruft Daten von dort ab. Kein lokaler Empfangsservice im WR-Code erkennbar.

Nulleinspeisung (0x1125)

Drosselt Einspeisung auf 0W Netzexport. Benötigt angeschlossenes E-Meter.

Frame-Bau (Python-Referenz)

```
import socket, struct, time

FRAME_START = 0x68
FRAME_END   = 0x16
CS_SEED     = 0x55

def checksum(data):
    return (sum(data) + CS_SEED) & 0xFF

def build_frame(cmd, serial, payload=b''):
    total_len = 12 + len(payload)
    body = (bytes([FRAME_START])
            + struct.pack(">H", total_len)
            + bytes([FRAME_START])
            + struct.pack(">H", cmd)
            + struct.pack(">I", serial)
            + payload)
    return body + bytes([checksum(body), FRAME_END])

def encode_tx(frame):
    """ASCII-Hex-Encoding für TX."""
    return frame.hex().upper().encode("ascii")

def receive_frame(sock, timeout=3):
    """Empfängt einen Binär-Frame vom WR."""
    sock.settimeout(timeout)
    # Startbyte suchen
    while True:
        b = sock.recv(1)
        if not b:
            return None
```

```

        if b[0] == FRAME_START:
            break
    # Länge lesen
    len_bytes = recv_exact(sock, 2)
    total_length = struct.unpack(">H", len_bytes)[0]
    # Rest lesen
    rest = recv_exact(sock, total_length - 3)
    return bytes([FRAME_START]) + len_bytes + rest

def recv_exact(sock, n):
    data = b""
    while len(data) < n:
        chunk = sock.recv(n - len(data))
        if not chunk:
            raise ConnectionError()
        data += chunk
    return data

```

Minimalbeispiel: Live-Daten lesen

```

SERIAL = 0xA1B2C3D4
IP = "192.168.1.100"

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.settimeout(5)
sock.connect((IP, 14889))

# Anfrage senden
req = build_frame(0x1077, SERIAL, bytes(20))
sock.sendall(encode_tx(req))

# Antwort empfangen (0x1051, 150 Bytes)
frame = receive_frame(sock, timeout=3)
cmd = struct.unpack(">H", frame[4:6])[0]
assert cmd == 0x1051

# 4 MI-Kanäle parsen (je 32 Bytes ab Offset 20)
for i in range(4):
    block = frame[20 + i*32 : 20 + (i+1)*32]
    uid = struct.unpack(">I", block[0:4])[0]
    dc_v = struct.unpack(">H", block[6:8])[0] * 64 / 32768.0
    ac_w = struct.unpack(">H", block[8:10])[0] * 512 / 32768.0
    kwh = struct.unpack(">I", block[10:14])[0] * 4 / 32768.0
    temp = struct.unpack(">H", block[14:16])[0] * 256 / 32768.0 - 40.0
    ac_v = struct.unpack(">H", block[16:18])[0] * 512 / 32768.0
    freq = struct.unpack(">H", block[18:20])[0] * 128 / 32768.0
    print(f"MI {i}: {ac_w:.1f}W {dc_v:.1f}V {ac_v:.1f}V {freq:.2f}Hz {temp:.1f}°C {kwh:.2f}kWh")

sock.close()

```

Offene Punkte

Bereich	Status
0x1175 (Get Power %) antwortet nie	Bestätigt
0x1006 Bytes 11–29 unbekannt	Nur Byte 10 (immer 0x00) und Byte 14 (Watt-Code) verifiziert
0x1137 Read-Modus existiert nicht	Bestätigt (gibt 0xef zurück)
E-Meter Datenweg (Cloud vs. lokal)	Wahrscheinlich Cloud, nicht abschließend geklärt
0x1041 niedrige Erfolgsrate (~20%)	Bestätigt – Retries zwingend

Quellen

Quelle	Details
APK	EnverView 4.1.20, decompiliert mit jadx
PCAP	PCAPdroid-Mitschnitte
Live-Tests	protocol_verify2.py gegen EVT2000SE (FW 164.125)
Portal	envertecportal.com